

RESEARCH

Open Access



Adaptive load balancing and fault-tolerant microservices architecture for high-availability web systems using docker and spring cloud

Pan Zhang¹, Lixia Xiang², Zhuo Song³ and Yuhong Yang^{3*}

*Correspondence:

Yuhong Yang

y-y8563@students.uc-bcf.edu.ph

¹Nanning University,
Nanning 530299, China

²Guangxi Science & Technology
Normal University, Laibin
546100, China

³CITCS University of the Cordilleras,
Baguio 2600, Philippines

Abstract

This study proposes a novel microservices architecture integrating adaptive load balancing and multi-level fault tolerance strategies. By combining Spring Cloud components with Docker containers, the design introduces three high-availability mechanisms: Eureka Health Check, Eureka Cluster, and Application Service Cluster. Experimental validation demonstrates a 20% QoS improvement over monolithic architectures (ZK, LAMP) and outperforms baseline Spring Cloud implementations in throughput (2942 requests/min) and fault recovery time (<5s). The framework addresses modern industry demands for scalability and resilience, validated through rigorous JMeter testing and AHP-based QoS evaluation.

Keywords Docker containers, Eureka, Microservices, Quality of service, Spring cloud, Web system architecture

1 Introduction

The evolution of web applications and growing demand for scalable, flexible, and high-performance systems have driven the implementation of microservices architecture over conventional monolithic structures. In terms of Internet applications, web systems have been used and popularized on a large scale by enterprises and developers because of their high efficiency, flexibility, ease of development, and low cost. Technology should evolve as the business grows. When the business is simple, monolithic architecture is a good choice because it is fast to develop and cost-effective [1]. However, with the emergence of more complex business scenarios, users increase, user data also increases, and service requests become slower and slower, which leads to a terrible user experience. The monolithic application architectures, such as MVC and SOA, are widely used in different business scenarios. Compared with a monolithic architecture, SOA can meet the characteristics of high coupling, high deployment efficiency, and security of architecture business. When business scenarios become complicated, SOA often cannot meet requirements due to its granular division and blurred service boundaries. Web applications that relied on monolithic architectures are facing significant challenges, including



© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

poor scalability, low system isolation, and high maintenance costs. As businesses grow and their systems become more complex, conventional architectures find it difficult to handle increased user data, service requests, and the need for efficient resource allocation.

Microservices architecture was coined by Martin Fowler in 2014 and has reformed how modern applications are developed and deployed. This architectural style decomposes applications into smaller, independent services that can be deployed and scaled individually [2]. These services interact with each other through lightweight protocols such as REST, which makes them flexible and scalable. Microservices resolve many of the limitations of monolithic and service-oriented architectures (SOA) in terms of system maintainability, scalability, and agility. Microservices permit independent development, deployment, and scaling of each service by decomposing applications into fine-grained services, leading to improved Quality of Service (QoS) [3].

Despite its many advantages, adopting microservices architecture presents several challenges. One main challenge is ensuring high availability and reliability against network failures, service disruptions, and communication overhead. These challenges are particularly crucial in distributed environments where the interaction between numerous services needs to be coordinated. Additionally, managing the deployment, scaling, and monitoring of microservices at scale requires strategies for service discovery, load balancing, and fault tolerance. The complexity of designing fault-tolerant microservices systems and managing service interdependencies remains a major issue for both researchers and experts.

One of the major challenges in this research field is the lack of comprehensive frameworks and strategies for optimizing QoS in microservices-based systems. While microservices provide flexibility, scalability, and modularity, achieving consistent QoS across all services requires careful consideration of factors such as latency, availability, throughput, and reliability. Moreover, there is limited research on benchmarking microservices architectures against traditional architectures to quantify the performance improvements in real-world scenarios.

In this context, this research focuses on designing a robust microservices architecture that can improve QoS in web applications. This research proposes an architecture using Spring Cloud components to achieve high availability and reliability. It also makes use of Docker containers to streamline deployment and enhance system efficiency. The architecture is evaluated against traditional monolithic systems using tools like JMeter, demonstrating that microservices architecture can improve QoS by approximately 20%.

This study explores the development of a fault-tolerant microservices architecture by integrating Docker and Spring Cloud and proposes a real-time adaptive load balancing strategy based on live server health metrics. The research addresses the challenge of improving the Quality of Service (QoS) in dynamic cloud environments by enhancing reliability, availability, and performance in microservices-based web systems. To validate the effectiveness of the proposed model, the system is empirically tested and evaluated against conventional monolithic architectures and modern orchestration tools. This research work aims to contribute an innovative solution to improve system responsiveness, fault recovery, and deployment efficiency in real-world scenarios through this approach.

The major contributions of this research are as follows:

1. Proposing a robust microservices architecture to enhance the QoS of web systems.
2. Implementing high availability strategies such as Eureka Health Check, Eureka Cluster, and Application Service Cluster to ensure service reliability.
3. Deploying the architecture using Docker containers for increased scalability and deployment efficiency.
4. Providing a benchmark comparison between microservices and traditional architectures, demonstrating considerable improvements in system performance and QoS.

The novelty of the proposed design can be identified through four major improvements over existing models. First, the model presents dynamic load balancing using real-time metrics to address the limitations of Spring Cloud implementations that depend on round-robin or static-weight load balancing. The system ensures more efficient workload distribution and faster response times by including live CPU and memory utilization data. Second, we propose a three-tier high availability mechanism that goes beyond basic failover techniques. This includes Eureka health checks for detecting unresponsive services, synchronized Eureka cluster configurations to eliminate single points of failure, and application-level service clustering for business logic failover. These layers form a robust fault-tolerance framework capable of reducing service recovery time and maintaining service availability during partial system failures. Third, we apply a quantitative QoS evaluation using the Analytic Hierarchy Process (AHP) to assess performance metrics and compare our architecture with other systems such as ZK and LAMP. Finally, we conduct empirical validation through containerized deployment of the system using Docker on Alibaba Cloud. Monitoring and analysis tools such as JMeter, Grafana, and Prometheus were used to evaluate performance. The results show 20% improvement in QoS to demonstrate the effectiveness of the proposed architecture.

The structure of this manuscript is as follows: Sect. 2 reviews the evolution of web system architectures and the key components of microservices. Section 3 presents the design of the proposed microservices architecture, including the use of Spring Cloud and Docker. Section 4 discusses the QoS evaluation methodology and presents the empirical results. Section 5 concludes the paper with a summary of the research findings and suggestions for future work.

2 Related work

Microservices design has become an effective method for enhancing the maintainability, scalability, and overall system performance. This literature review elaborates on the evolution of web system architecture, key contributions in the field, addressing the challenges and advancements of microservices in the context of improving quality of service (QoS).

2.1 Web system architecture evolution

The web system has experienced a long development process since its beginning in 1998. From the initial interface display system to the later functional system, the architecture of the web system is continually evolving. From the beginning of development to the present, it has mainly gone through three phases: monolithic architecture, SOA, and microservices architecture [4].

2.1.1 Monolithic architecture

Monolith is the initial state of the web system architecture. It has only one code base, and deployment and scaling are based on system modules. Therefore, monolithic architectures became popular among SMEs due to their high efficiency and fast and simple style characteristics. The monolithic architecture is easy to develop and deploy due to its small size. For example, Netflix, Amazon.com, and eBay, which are very well-known Internet platforms all over the world, were initially simple web applications [5]. As the application grows and the project needs to be upgraded and expanded, the monolithic architecture faces many problems, such as web container reload, IDE reload, and difficult continuous deployment due to the disadvantages of being difficult to maintain and confusing to expand [6].

2.1.2 Service-oriented architecture

To solve the problem of information isolation between subsystems in system expansion, Service-Oriented Architecture (SOA) was born. SOA is a component model, which aims to integrate the different functional units of the system into services, and these services are connected through interfaces. Many developers agree that SOA is a distinct system decoupling method and a coarse-grained, relatively loosely coupled system architecture [7]. Communication between different services is achieved through pure and precisely defined interfaces, and the maintainability and scalability of the entire system are improved compared to the monolithic architecture. Compared with the monolithic architecture, although SOA is systematically divided, when the number of services increases, it is difficult to assess the service capacity accurately. Besides, many subsystems cause a waste of resources during actual operation [8].

2.1.3 Microservices architecture

The concept of microservices has been widely discussed and disseminated recently, and it is used to describe an independently deployed software application design method. This architecture is not very precisely defined, but it has significant features in terms of business capabilities, automatic deployment, service splitting, and data dispersion [9]. Newman [10] introduced some specific practices of microservices architecture based on the microservices theory. Beyer [11] discussed the development status of microservices from the microservices model and particular applications. The most relevant and similar work to this thesis approach is the research work of Kai [12], in which he designed a training system using the framework of Spring Cloud. In this paper, he only gives a straightforward application of the microservices framework and does not provide relevant measures for system improvement.

2.2 Frameworks in microservices

A framework is a set of classes that work together to build a reusable design for a specific type of software [13]. In short, a framework is a collection of programs that can be reused in particular application development. The frame gives architecture, dependencies, and task flow during application development.

2.2.1 The main frameworks in microservices

The gRPC is a high-performance open-source RPC framework developed by Google that can build a simpler and more unified API layer. Thrift is a cross-language service deployment framework that can be understood as a C/S model. It generates server-side and client-side code from the interface definition file through a code generation tool, thereby achieving cross-language support for the server and the client [14].

From the perspective of the overall architecture in Table 1, Dubbo and the Spring Cloud framework are the closest. Both of them need a service provider, a registry, and a service consumer. However, Dubbo only implements service governance, and Spring Cloud covers all aspects of the microservices architecture. From the perspective of function expansion, as long as the Spring Cloud subproject is integrated during the development process, the integration of various components can be completed. Dubbo lacks the need to implement multiple filters to customize, and the development cost and technical difficulty are slightly higher. Therefore, the microservices architecture proposed in this web system uses Spring Cloud as the development framework.

2.2.2 Spring cloud

Spring Cloud is a complete set of microservices solutions based on the Spring Boot framework. It integrates excellent framework features in the field of software architecture to rapidly build distributed systems. It has direct access to related tools, which simplifies the amount of code for developers [15]. Spring Cloud includes a series of functions such as service registration and discovery, routing network management, load balancing, and cluster monitoring. It can implement a fast and reliable microservices system. The characteristics of the spring cloud are summarized as follows.

2.2.3 Spring boot

Spring Boot is a new framework provided by the Pivotal team and has a proper application in microservices [16]. One of the features that makes Spring Boot very popular with developers is that it simplifies the original technology and can directly import the corresponding functions into the developed modules. Use Spring Boot in the applications to run projects faster and use as few complex configurations as possible. Spring Boot includes a lot of function libraries, which can be used by the build system of any project. The characteristics of Spring Boot are summarized as follows [17, 18].

Table 1 Comparison of main frameworks in microservices

Framework/Function	Motan	gRPC	Thrift	Dubbo	Spring cloud
Functional positioning	RPC Framework	RPC Framework	RPC Framework	Service Framework	Microservices Framework
Rest (yes/no)	No	No	No	No	Yes
RPC (yes/no)	Yes	Yes	Yes	Yes	No
Multi-language	No	Yes	Yes	No	Yes
Service/registrar discover	Zookeeper	No	No	Zookeeper/	Eureka
Load balance	Yes	No	No	Yes	Zuul/Ribbon
Configuration Service	Yes	No	No	No	Spring Cloud Server
Service call	No	No	No	No	Zuul/API
Fault tolerance	Yes	No	No	Yes	Hystrix/Ribbon
Typical Case	Sina	Google	Facebook	Alibaba	Netflix

2.3 ZK and LAMP framework

In the research of Liu [19], a web system design based on the ZK framework is given. It is a typical application of an MVC framework. ZK is primarily composed of two parts, and one is the server based on AJAX background processing technology. During the design and implementation of web software systems based on the ZK framework, it is easy to conduct the deployment [20]. In the system design of Liu [21], the front-end and layered back-end designs were implemented using the MVC model. In the system design, the Spring Framework technology is used together because there are many powerful components in the Spring Framework [22]. The use of ZK technology does not require JavaScript, and interface creation and event response can be achieved only through ZUML. There are also some shortcomings in the design of the MVC pattern system based on ZK technology.

The LAMP architecture method is a typical platform architecture method for building high-performance web application systems. Developers can build their modules to handle specialized tasks, such as implementing identity authentication in Apache through loadable modules [23]. In the research of Wang [24], a web system based on LAMP (Linux + Apache + MySQL + PHP) architecture was constructed. In the system design, Nginx was introduced to improve the architecture deployment scheme, and load balancing technology was used to slow down the server's pressure on high concurrent access and achieve system stability.

2.4 Docker container

Docker is an open-source platform for deploying applications in software systems. It allows developers to package their applications and dependencies into a lightweight, portable container and then publish it to any Linux machine for virtualization [25]. Docker is a Linux-based container management solution. It uses process communication technology to isolate the file resources of the operating system from the network. Use Docker to package, test, and deploy applications faster, and shorten the cycle from writing to deploying and running code. In Docker containers, resource isolation is implemented through namespaces to ensure that vessels do not affect each other. Meanwhile, we also checked some papers who compare with Kubernetes-based solutions, service meshes (Istio), and QoS-aware frameworks [26]. We observed the lack of integrated health checks and adaptive load balancing in Spring Cloud.

2.5 Modern approaches of microservices and applications

The implementation of microservices architecture has become increasingly significant in cloud-native system design due to its modularity, scalability, and flexibility. Several researchers have explored how microservices architecture can improve system reliability, quality of service (QoS), resource utilization, and real-time adaptability across various domains. Recent studies have also examined how advanced orchestration, scheduling algorithms, and integration with machine learning can optimize microservices performance. This section reviews existing works related to microservices-based QoS enhancement, fault tolerance, adaptive scheduling, and performance evaluation across several environments.

Madnani et al. [27] studied the architecture of automated scoring systems, emphasizing the importance of reliability, scalability, and flexibility in production environments.

They highlighted that microservices architecture can support these requirements due to its modular design, enabling efficient development by multiple teams. This is important when integrating multiple programming languages and maintaining high system stability, particularly for real-time services like automated scoring. Their work provides a foundation for understanding how microservices can enhance reliability and scalability in complex applications. Bhamare et al. [28] discussed the growing complexity in accommodating increased service demands brought by advancements in networking, such as 5G and IoT. They emphasized the prevalent adoption of microservices by major organizations like Netflix and Facebook to address these challenges. However, they also noted a lack of standard guidelines for service providers on deploying microservices, mainly in handling user-level QoS requirements like latency and cost. Their work introduced an affinity-based scheduling heuristic to optimize microservice deployment which achieved significant improvements in service efficiency and cost-effectiveness.

Al-Masri [29] emphasized the importance of QoS in Industrial IoT (IIoT) applications. They proposed a quality-aware microservices architecture that continuously monitors services to ensure functionality. This contribution is significant as it emphasizes real-time monitoring of QoS attributes which is essential for dynamic and resource-constrained environments like IIoT. Bushong et al. [30] categorized recent approaches to analyzing microservices systems. They observed that microservices have greatly improved deployment cycles and scalability, but they still face challenges in understanding system boundaries. The paper categorized five key analytical approaches to better understand the behavior and interactions of microservices. This work provided a roadmap for further research into tools and techniques for cloud-native systems.

Gao et al. [31] studied the reliability of microservices in cloud environments, focusing on fault-tolerant strategies for critical services. Their formal modeling of microservices and redundancy mechanisms provided a method to enhance reliability in cloud applications. This work is mostly relevant for improving QoS as it addresses service reliability and strategies to handle system failures. Söylemez et al. [32] conducted a detailed review of the challenges in adopting microservices architecture. Their work is essential for anyone considering MSA adoption, as it offers insights into potential pitfalls and solution directions. They stressed that while MSA offers numerous benefits such as modularity and flexibility, careful evaluation is required to achieve the expected QoS improvements.

Samodha et al. [33] focused on the role of microservices in fog computing. They studied QoS-aware application scheduling for IoT environments. They introduced the QMPSO algorithm for optimizing resource utilization while maximizing QoS parameters such as throughput and budget satisfaction. This research is important in the context of distributed computing environments, as it demonstrates how microservices can improve latency-sensitive and bandwidth-hungry applications by ensuring better resource allocation. Hossen et al. [34] addressed the challenge of resource management in microservices-based cloud applications. They introduced a resource management tool that optimized resource allocation through opportunistic reduction strategies. Their work demonstrated up to 33% resource savings and shows how efficient resource management can significantly improve the QoS of microservices systems.

Tokmak et al. [35] explored the use of machine learning (ML) techniques for improving the QoS of microservices through the use of service discovery tools that assess parameters such as efficiency, delay time, and response time. Their study used boosting

algorithms to predict microservice quality and achieves 90.42% accuracy in their experiments. This work is a key example of how ML can be integrated into microservices architecture to enhance QoS by dynamically predicting and optimizing system performance. Hassaan et al. [36] reviewed the existing microservices architecture for IoT systems with a particular focus on improving non-functional characteristics like reliability and availability. They identified several strengths in microservices' ability to address these challenges but noted that further research is required to overcome certain weaknesses in current approaches. Their work supports the potential of microservices to enhance the QoS of IoT systems whereas pointing out the need for ongoing innovation.

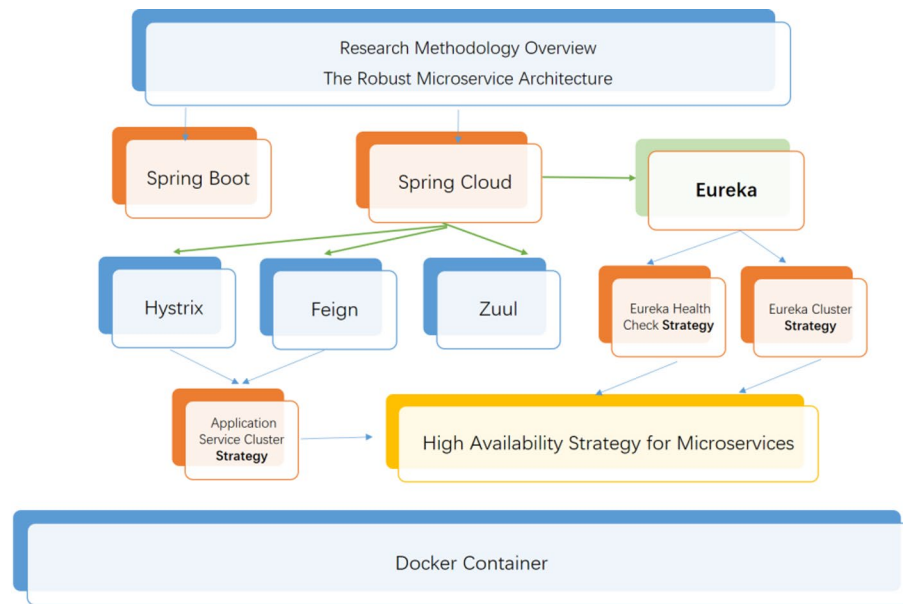
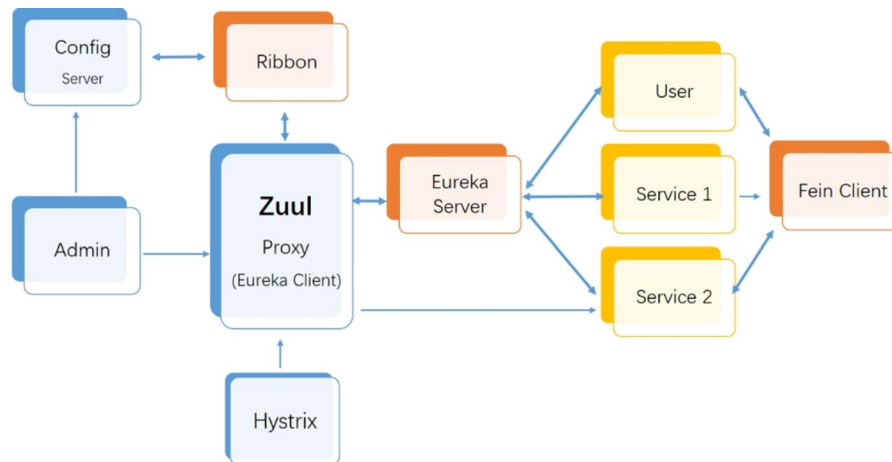
Alelyani et al. [37] proposed a new scheduling approach to address two major challenges in microservices architectures such as poor network traffic management and inefficient resource allocation. This work schedules microservices using Particle Swarm Optimization (PSO) algorithm. The load balancing mechanism is also achieved using round-robin algorithm. The experimentation is done in a simulated environment using Alibaba and Google datasets. Experimental outcomes demonstrates that the PSO-based approach manages network traffic, balances the load, and utilizes the CPU efficiently. Singh et al. [38] presented a review pertaining to the containerized environments for big data applications. They also proposed a scheduling mechanism for microservice architecture to balance the load. The experimentation is done on a single server and scaled to handle four instances. Experimental results show that the proposed architecture is effectively handle the loads using Docker Swarm.

Hossain et al. [39] done an in-depth review on benefits of microservice architecture in edge computing environments. They examined the research directions in the areas of orchestration, deployment, coordination, resource allocation etc. The applications of microservices in edge computing environments for various domains is also investigated. Dashtbani and Tahvildari [40] introduced an adaptive auto-scaling framework for microservices named STaleX. It amalgamates spatiotemporal features. It utilizes weighted Proportional-Integral-Derivative (PID) controllers for every service and tunes weights dynamically based on a supervisory unit that monitors service dependencies and workload variations. Experimental results show that the approach attains a 26.9% reduction in resource usage compared to traditional scaling methods. Barua and Kaiser [41] studied the application of microservices architecture in developing scalable and reliable airline reservation systems. It elaborates the use of circuit breaker patterns, load balancing and real-time monitoring to achieve high availability and fault tolerance. It results in significant improvements in system scalability and uptime.

3 Proposed model and system design

3.1 Proposed architecture overview

The proposed microservice architecture is depicted in Fig. 1. The entire system is based on a microservices architecture, which uses Spring Boot and Spring Cloud as a microservices framework. It can achieve rapid development and service governance. The three strategies of Eureka Health Check, Eureka Cluster, and Application Service Cluster can achieve the goals of high availability and high reliability of microservices. Finally, based on the container technology, the project is deployed in the Docker container to achieve rapid deployment and solve the problem of the high cost of operation, maintenance, and deployment.

**Fig. 1** Proposed microservice architecture**Fig. 2** Spring Cloud components access flowchart [42]

3.2 The robust microservices architecture design

3.2.1 Spring boot and spring cloud framework

Spring Boot integrates a large number of commonly used third-party library configurations, which can reach the feature out-of-the-box [42]. Spring Cloud components access flowchart is illustrated in Fig. 2. In most applications of Spring Boot, only a minimal amount of configuration code is needed to build a basic development framework, which enables developers to focus more on the implementation of business logic without spending much energy on the bottom layer. Spring Cloud is a microservices solution that provides components including Config, Eureka, Zuul, Hystrix, and Feign. Especially when the developers design a system, the reasonable use of related components can develop a more robust system. Eureka manages service discovery; Hystrix handles circuit breaking. Dynamic health checks with custom thresholds also beed defined in this paper (e.g., service marked “DOWN” if CPU > 80% for 10 s). Adaptive load balancing via

Feign/Ribbon with real-time server weight adjustments (CPU/memory-based). Eureka Cluster Sync Optimization: Reduced synchronization latency between Eureka nodes from 30 to 5 s.

Eureka is a service registration and discovery center. Eureka consists of a server and a client. Eureka service registration and discovery is used as the core component to realize dynamic monitoring of services and invocation of services. Service registration and innovation are vital parts of service invocation. Zookeeper and Eureka are the two most commonly used and most popular components of service registration and discovery components. Compared with Eureka, Zookeeper cannot focus on the availability of the service itself, and it is not suitable for application in the actual project [43]. Multiple Eureka servers are generally required to register with each other to form a cluster to achieve high availability. Eureka Server synchronization process, as long as the conditions for node connection are met, information can be transmitted and synchronized. Eureka's service registration architecture is depicted in Fig. 3.

Eureka can automatically call the corresponding service based on the service name. To ensure excellent performance, Eureka does not always access the registration center to query the service. When Eureka Client registers with Eureka Server, it will synchronize data with each other. Register and discover Eureka Server's services by making remote service calls with Feign. The service information in the Eureka Server list can be cached in the client, and it will be obtained from the registration center when the local service is unavailable. Eureka can reduce the pressure on the server and increase the time-of-service invocation. The Eureka service registration center completes system service registration and business processing through three nodes [44]. During the service registration process, the first function is to add the component *EnableDiscoveryClient* (an annotation) operation to the primary function to implement the *DiscoveryClient* instantiation. The second is to modify the service URL default zone parameter value in the configuration file.

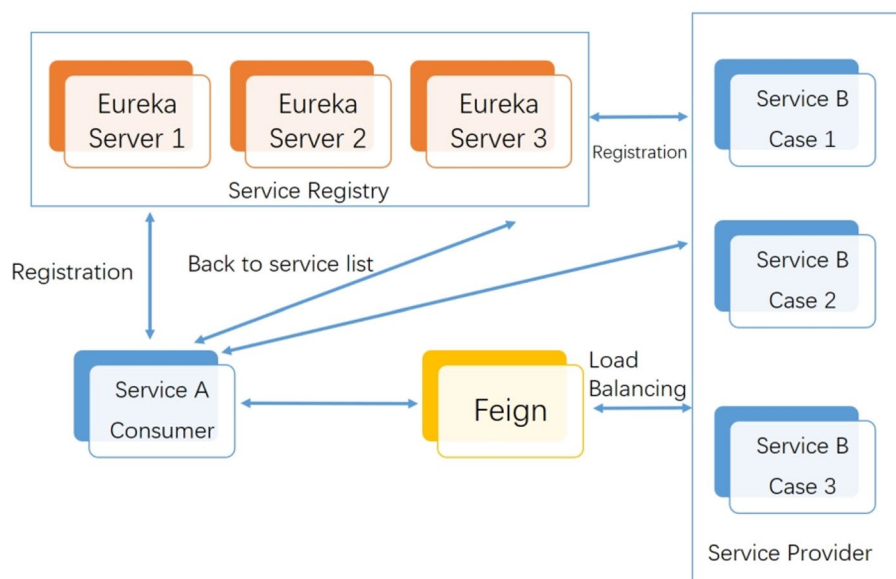


Fig. 3 Eureka's service registration architecture diagram

3.2.2 Spring cloud zuul

Zuul is a kind of gateway and router. It forwards requests to the corresponding services. The gateway is implemented in the system design by using Zuul. It provides functions such as dynamic routing and security filtering. When the system's concurrency is high, multiple Zuul nodes can be set to form a Zuul cluster. By configuring the properties file, different requests are forwarded to the corresponding services. The Zuul gateway workflow diagram is shown in Fig. 4.

3.2.3 Spring cloud hystrix

Hystrix is used for different service scheduling and isolation to avoid the problem of service avalanche. The service is split into individual services in distributed systems, and calls between services are implemented through Feign. When a network failure occurs, the call may fail, and the remote service may be unavailable. Once a large number of requests are sent at the same time, the system may cause a severe “avalanche” problem. By using Hystrix components, faulty services can be forcefully blown out, and related requests can be distributed or downgraded to prevent and control faults. Although microservices have the advantages of easy management, secure development, and rapid deployment, it should be noted that in the pursuit of agile development, the reliability of service invocation is an essential guarantee [45]. The Hystrix fuse architecture diagram is represented in Fig. 5.

3.2.4 Service failure handling in microservices

In a microservices architecture, when a service instance fails, the system follows a series of steps to ensure overall robustness and availability. The service failure handling is illustrated in Fig. 6.

These steps involve service discovery (Eureka), client-side load balancing (Ribbon), and fault isolation (Hystrix):

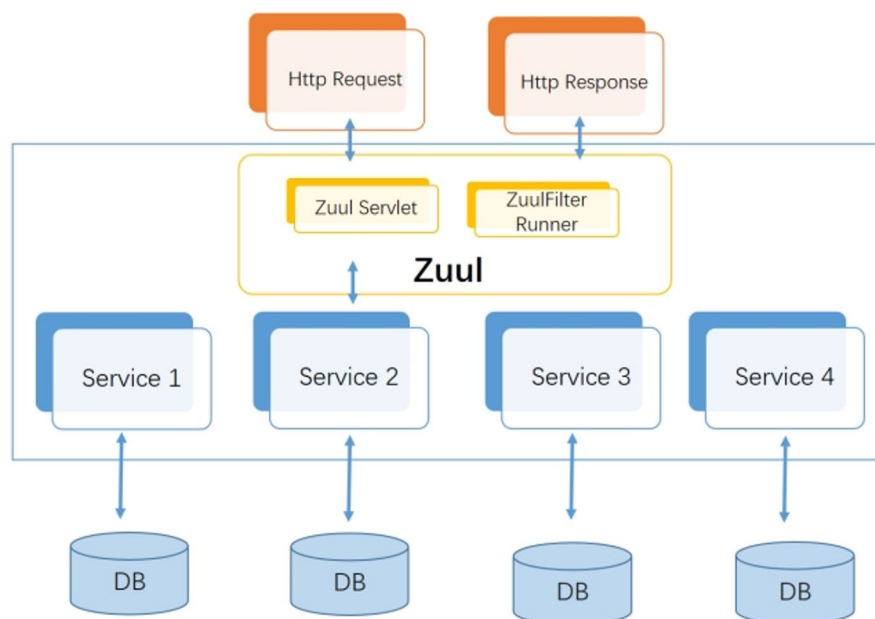


Fig. 4 Zuul gateway workflow diagram

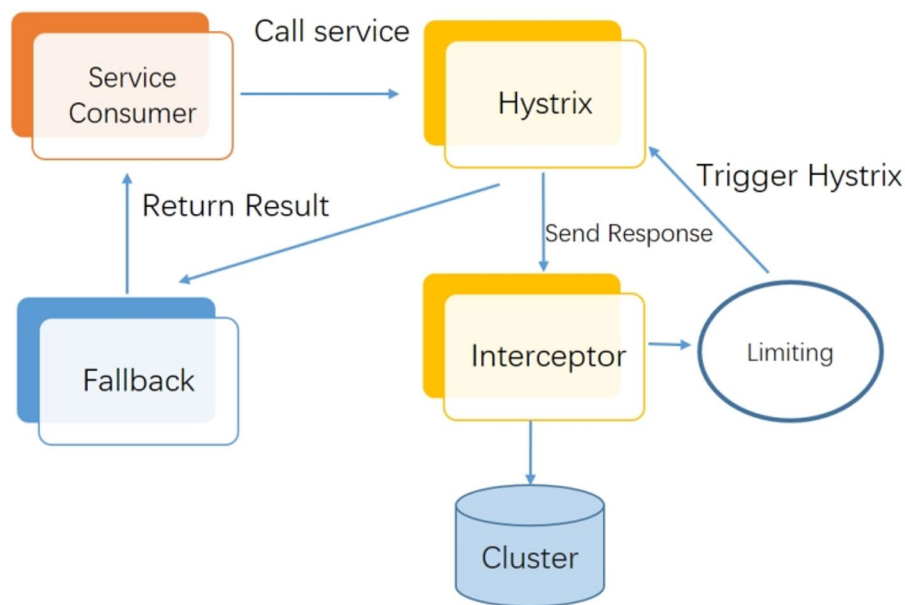


Fig. 5 Hystrix fuse architecture diagram

1. Service invocation: The client requests the target service by fetching its instance list from Eureka via Ribbon and selecting one according to the load-balancing algorithm.
2. Service failure: The chosen instance becomes unresponsive (timeout or exception).
3. Circuit breaker and fallback: Hystrix detects the failure, opens the circuit, and executes predefined fallback logic, returning a default response to the client.
4. Health check: Eureka marks the failed instance as DOWN after missing its heartbeat and removes it from the registry.
5. Load-balancing retry: Ribbon retries the request on another available instance.
6. Registry update notification: Eureka synchronizes the status change so that all service consumers are aware of the instance's unavailability and avoid calling it.

3.2.5 Spring cloud feign

Feign is a request call for services. In the microservices project, Feign and Hystrix together achieved load balancing. It is a fast and straightforward component with a declarative and modular HTTP client that calls the HTTP API in the spring cloud. In the process of its use, it can implement HTTP calls by creating interfaces and adding partial annotations. Feign uses interface-based annotations, which can be achieved through Feign annotations and JAX-RS annotations. When one service is killed, the other service continues to run without being affected, which can achieve system load balancing. Hystrix is also integrated to make it fuse. For example, if a service is found to be down, Feign will return a failure message so as not to cause concurrent waiting. When a request is initiated between services, the system performs load balancing based on Ribbon and allocates services. This is illustrated in Fig. 7.

3.3 High availability multi-level strategies for microservices

In this section, three levels of microservices high availability strategies, first-level strategy: Eureka Health Check Strategy, second-level strategy: Eureka Cluster and third-level

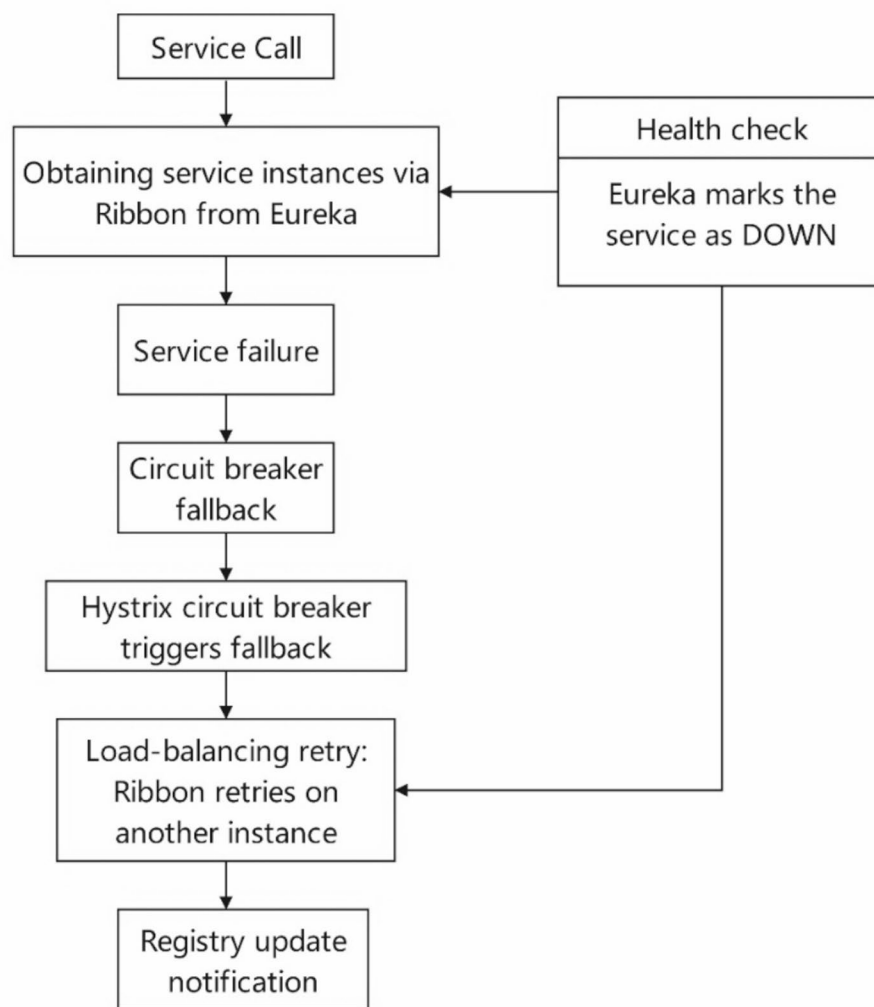


Fig. 6 Service Failure Handling Workflow in Microservices Architecture

strategy: Strategy and Service Cluster Strategy, are proposed to achieve the purpose of completing a robust microservices architecture.

3.3.1 Eureka health check strategy

In the microservices system, each sub-business system can be registered in Eureka to realize service registration and discovery. The essential functions of Eureka service registration have been discussed in the previous section. In microservices, configure Eureka server parameters and interfaces on the server side, set the self-protection mode to false, and modify the inspection effectiveness time. Through these settings, it can implement the inspection of registered services in Eureka, and perform the inspection of Eureka services.

Service failure is often encountered during system operation. Eureka has a health check mechanism. By modifying the program to increase the configuration, the feedback signal sent by the client can be detected in Eureka, and the activity of the service in the microservices can be determined. Accurately identify some error services in Eureka that cannot continue to run and mark the error services as "Down." Eureka health check means that Eureka monitors the services registered in the system. The feedback

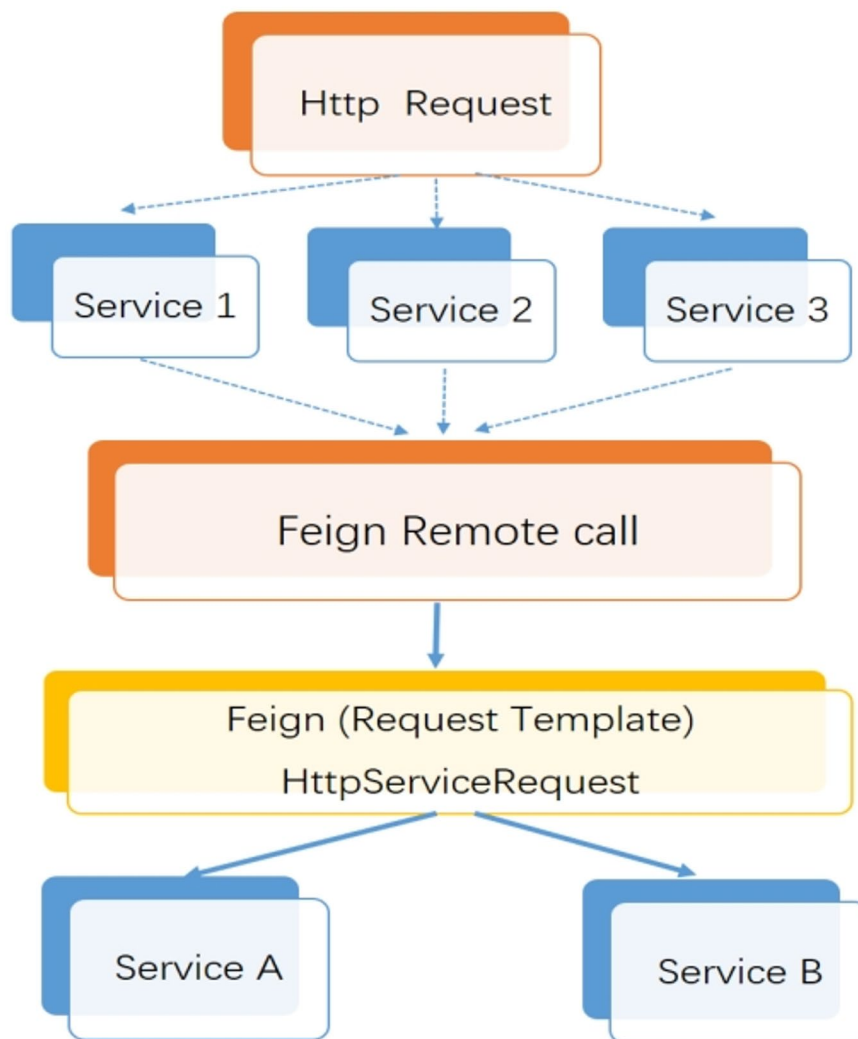


Fig. 7 Feign service remote call

mechanism between the Eureka server and the Eureka client determines the status of the Eureka client through "operation status signal feedback." By default, the status signals of the server and the client are healthy, and the service remains in the "UP" state. *Read-WriteMap* and *ReadOnlyMap* are two instance objects in the Eureka server.

After registration, when the service client starts the heartbeat service synchronously, it will establish a monitoring mechanism with the Eureka server in time. Eureka will receive the expiration time of the service, and at this stage, it will periodically check each service. Figure 7 shows the specific Eureka health check process. When the service cannot be updated in time, the expired service will be automatically deleted in *Read-WriteMap* to monitor the service. After updating the service registration, Eureka will synchronize the service list to the local cache and send the information to each service to complete the health check. The Eureka health check is summarized in the following four steps:

1. The Eureka client uses register and passes it to the Eureka server via an HTTP request to achieve the transmission of system service information. When service providers and

- consumers coexist, perform health monitoring through the "heartbeat mechanism," which is the transfer of status information.
2. The server receives feedback from the client within the corresponding time; the client is in a healthy state and marked as "UP." If no input or erroneous feedback is received within an appropriate time, it indicates that the client is not healthy; mark it as "Down," and the service removes the service list.
 3. The user can maintain the service instance by setting the fetch-registry: false method instead of retrieving the instance.
 4. Through the Spring Cloud component Feign or Ribbon, users can re-enable available services to send requests.

This process is illustrated in Fig. 8. In short, by designing and implementing the Eureka Health Check strategy, the service failure can be checked in the microservices system. Moreover, the faulty service can be eliminated promptly to ensure that the services of the registry can operate normally and achieve high availability of microservices.

3.3.2 Eureka cluster strategy

In the previous section, we elaborated on the first-level strategy Eureka health check function. It raised a question in the end on how to achieve Eureka's high availability. There are two methods for cluster strategy. The first method is to make a backup of the registration service in Eureka to achieve the goal of high availability. When Eureka goes down, the backup service can continue to run by recalling. Then, after specific practice, it will find that this method is not operable and has high complexity. Therefore, the second method to build a Eureka Cluster became the right choice. As shown in Fig. 9, when designing the microservices architecture system, set three Eureka services and name them Eureka7001, Eureka7002 and Eureka7003 at the same time. Moreover, Eureka7003 instances are registered with eureka7001 and eureka7002, respectively, to ensure the

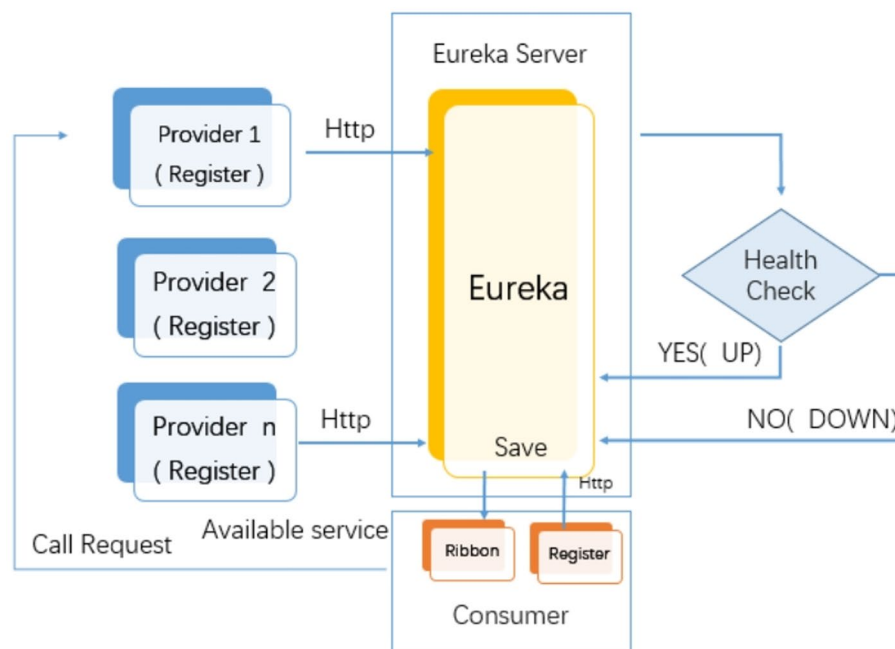


Fig. 8 Eureka health check process

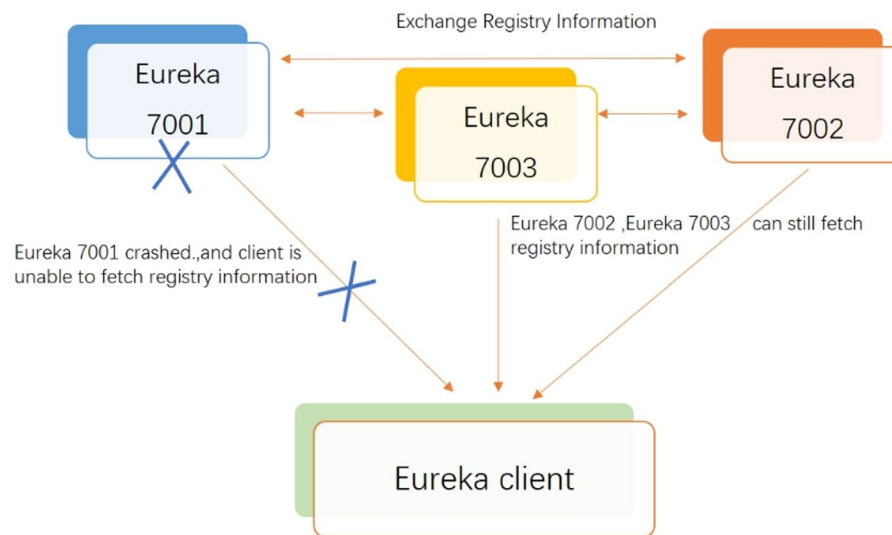


Fig. 9 Eureka cluster model design

synchronization of information to achieve cluster mode. Service registration information can be exchanged among these three Eureka services, and each Eureka can fetch registry information from other Eureka services. As a result, it ensures the synchronization of the service registry.

From the design of the Eureka cluster mode, Eureka's automatic failover can be achieved. When a service failure occurs on a Eureka7001, Eureka will internally start a port to call the error mechanism automatically, and the registered service information will be exchanged and updated promptly. The service is automatically migrated to another Eureka7003 and Eureka7002. During the entire switching process, the call between services is transparent and fast. In the process of building a Eureka cluster, it can use the server cluster. In other words, if different Eureka7001, Eureka7002, and Eureka7003 are deployed on different servers if one Eureka server crashes. Eureka on other servers can continue to run, thus ensuring that clients can still connect to the remaining Eureka servers and discover other services. In this microservices project experiment, the complexity of the project is lower, the access pressure is small, and the hardware conditions and other comprehensive factors are also affected. Therefore, they are just deployed on one Alibaba Cloud server to implement Eureka clusters. Eureka's cluster strategy can realize the backup of multiple services, thus achieving high availability of Eureka and further enhancing the high availability of the microservices system. The design of the Eureka cluster mode is an excellent high-availability strategy for this project. Eureka's high availability is achieved through the Eureka cluster. At the same time, imagine if it was not Eureka that failed, but the service in the microservices system failed. Here is another strategy, called Microservices Cluster Strategy. It can solve the problem caused by the microservices application service's failure to achieve high availability. Eureka cluster model design is demonstrated in Fig. 9.

3.3.3 Application service cluster strategy

The Eureka cluster realizes the function that the regular system operation continues to run when Eureka breaks down. In some actual and big projects, it will encounter

the problem of application service failure. To further enhance the high availability of microservices, by simulating the Eureka cluster, the application service cluster is applied to achieve the purpose of microservices high availability in microservices application services. Specifically, it means that an application service fails, and other services can be replaced to make the corresponding functions without affecting other application services.

Eureka service registration and application service cluster can solve the problem of service failure cascading, that is, when a service fails, it will not cause a series of related services to have problems. In this microservices project, because the business is not complicated and affected by comprehensive factors such as hardware conditions, the application service is deployed on one server as well. In the microservices project, three different microservices application services were built, microservices-userService-8001, microservices-userOrderservice-8004, and microservices-userProductservice-8006. At this time, if there is a problem with userProductservice, it will result in no such user-Productservice application service in the system. In other words, this service is not available anymore. Therefore, build one more group of application services with the same service functions as microservices-userService-8002, microservices-userOrder-service-8005, and microservices-userProductservice-8007. During system operation, if microservices-userService-8001 fails, then microservices-userService-8002 can be called and used by consumers. Figure 10 shows how to implement the service cluster function in microservices.

The application service cluster solves the problem of cascading service failures faced by a single service. It is also known as a monolithic application. By using the application services cluster, it realizes automatic failover of service exceptions, thereby achieving high availability of services. In other words, a service provider hangs up, and other service providers with the same function can take over the failed service and ensure the regular operation of the system.

3.4 Containerization with docker

Here, the Docker container is introduced to implement the microservices system deployment to achieve lightweight virtualization. The Docker operation is exemplified in Fig. 11. Using Docker can achieve rapid expansion of services and flexible

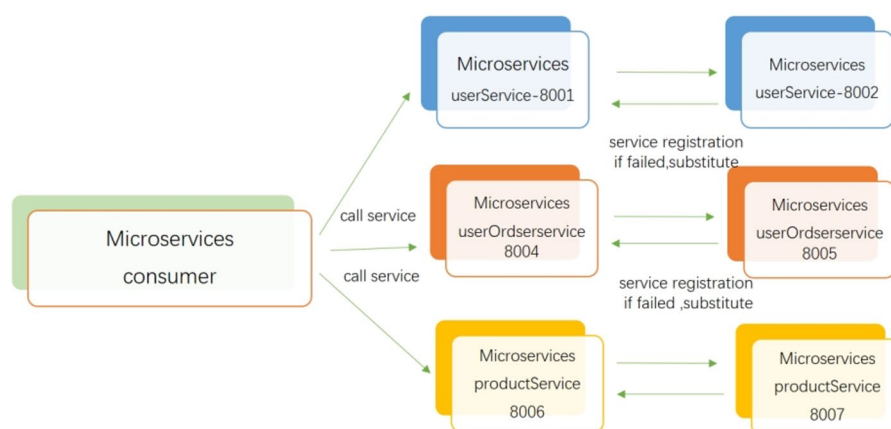


Fig. 10 Application services cluster function

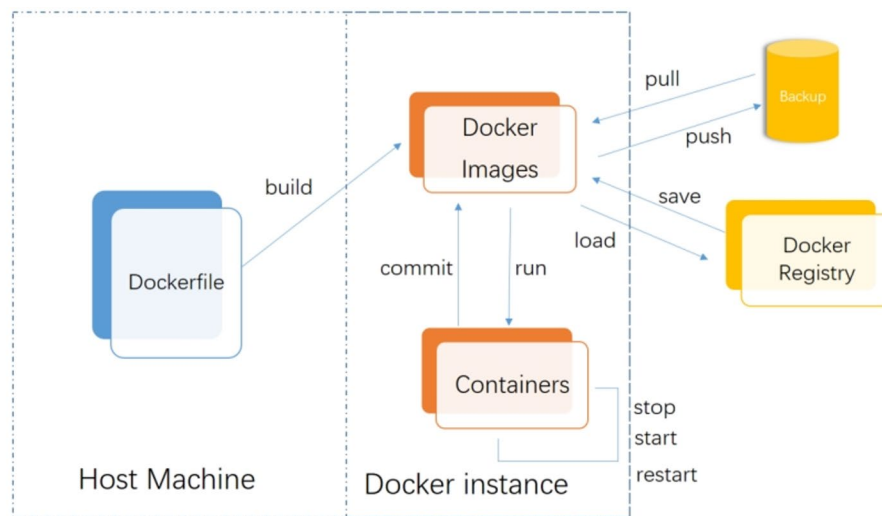


Fig. 11 Docker operation flowchart

implementation of services. Each function in the system and its dependencies are packaged in each container to achieve service isolation. Using Docker containers can also solve the impact of the external environment on the system, which can also continue to test, deploy, and run the project.

3.5 QoS evaluation method

QoS is a comprehensive indicator to measure system performance. However, there is currently no comprehensive and accurate QoS evaluation method for web systems, especially for web applications such as microservices systems. In this section, the QoS Evaluation Approach is proposed in conjunction with the *Analytic Hierarchy Process* (AHP), which can better comprehensively evaluate the system [46]. In the field of software systems, there are many methods for evaluating QoS. However, there is no standard for the specific QoS measurement of microservices. Most QoS evaluation methods only use a single QoS parameter, such as throughput or transaction time, as the measurement standard. More parameters are considered here to evaluate the QoS more comprehensively by using the throughput [47], response time, error rate, reliability [48], availability, resource Utilization rate and aggregate report data [49].

The analytic hierarchy process is a method for analyzing complex decision-making, which is a combination of qualitative and quantitative methods [50, 51]. The analytic hierarchy process uses the idea of hierarchical decision and hierarchical description and has the practicality and efficiency of solving complex problems [52, 53]. The specific purpose of the AHP is to score all decisions or conclusions. During the scoring process, the scores are graded according to different vital levels from level 1 to level 5 with different level descriptions like no effect, low, medium, high, and very high [54]. The scores count from 1 to 4, and the calculated result is used as a score to measure the QoS of the system [55, 56]. The higher the score, the better the performance of the item, and the higher the final result score, the better the system performance based on the architecture [57].

3.6 Modeling of load balancing and QoS evaluation

In this work, the server selection mechanism is done using a weighting function. Here, weight each sever is computed based on real-time CPU and memory usage. The selection probability P_i for sever i is modeled as:

$$P_i = \frac{w_i}{\sum_{j=1}^n w_j} \quad (1)$$

where $w_i = \frac{1}{\alpha.CPU_i + \beta.Memory_i + \epsilon}$

Here, α and β are normalization coefficients, and ϵ is a small value that is used to prevent division by zero. This function lowers the selection probability of overloaded servers dynamically.

The Analytic Hierarchy Process (AHP) structure has been elaborated mathematically using the pairwise comparison matrix A, where consistency ratio (CR) is calculated as:

$$CR = \frac{CI}{RI} \quad (2)$$

where $CI = \frac{\lambda_{max} - n}{n - 1}$

and λ_{max} is the maximum eigenvalue of matrix A, and RI is the random consistency index. AHP starts by comparing each QoS criterion with others using a scale of relative importance. For example, if response time is moderately more important than error rate, we assign a 3 in the matrix. Each column of the matrix is normalized by dividing each element by the column sum. The weight vector w is calculated by averaging the rows of the normalized matrix. This gives us the relative importance of each parameter. The Consistency Index (CI) and Consistency Ratio (CR) are computed to verify logical consistency in judgments. If $CR < 0.10$, the judgments are considered consistent.

4 Results and evaluation

In this section, we present the empirical results and evaluation of the proposed robust microservices architecture. The performance of the system was tested using the JMeter tool to measure key Quality of Service (QoS) parameters, including response time, throughput, error rates, and system stability. The results are analyzed in comparison to traditional monolithic architectures, such as ZK and LAMP, to demonstrate the significant improvements achieved through the microservices-based design. Runtime Performance is 65% usage of CPU, which under 1000 threads with memory usage: 1.2 GB/ service at peak load.

4.1 Empirical results and analysis

For this experiment, the test Application is a self-developed website, which included 6 Microservices:3 Spring Cloud components: Eureka Server, Zuul Gateway, Hystrix Dashboard and 3 Business services: UserService, OrderService, ProductService (each with REST APIs). As for deployment:Using Docker containers on Alibaba Cloud (8 vCPUs, 16 GB RAM). Monitoring Tools: Grafana for CPU/memory metrics and JMeter Aggregate Report for throughput, error rates, and latency.

The microservices system was designed with multiple high-availability strategies, including Eureka Health Check, Eureka Cluster, and Application Service Cluster to ensure robustness. These features were tested thoroughly, and the performance metrics

were compared with the expected outcomes. This section mainly expressed the detailed test results and analysis of the previous microservices experiment. At the same time, there are comprehensive analyses and comparisons of microservices and monolithic architecture, and it will also show the results of solving research problems. The adaptive load balancing mechanism dynamically adjusts traffic distribution based on real-time server health metrics. Each server's **selection weight** is inversely proportional to its current resource utilization (CPU, memory) and latency. Servers with lower combined resource consumption and faster response times receive higher priority in request allocation. For instance, if a server exhibits high CPU usage or memory pressure, its weight decreases, reducing its likelihood of being selected for new requests. This ensures efficient resource utilization and minimizes latency. This approach contrasts with static load-balancing algorithms by incorporating real-time performance data, enabling proactive adjustments to workload distribution and enhancing overall system resilience.

4.1.1 Microservices function and performance analysis

Table 2 shows the key functions of the microservices system and their performance during testing. Each component of the architecture, including service registration, load balancing, and fault tolerance, was tested under various conditions to ensure high availability. It can be seen from the table that the microservices project was built successfully. It achieved the high availability performance of microservices and solved the common problems of the monolithic architecture. From the comparison result of traditional monolithic architecture, a service failure may cause the entire system to fail. In the high availability test of the Eureka cluster, application service cluster and load balancing have verified that high availability can be achieved by the microservices architecture, and it improved the QoS of the system.

4.1.2 Microservices QoS data analysis

In this section, the data collected from the JMeter testing tool, postman, and other system testing tools will be shown and analyzed. According to the proposed QoS parameters and QoS calculation method, it is also discussed in this section.

(i) Response Time (ms).

The response time of the system is an important factor in evaluating user experience. As shown in Fig. 12, the average response time remained stable during the test phase, even

Table 2 Microservices function and performance test results

Microservices function	Expected result	Passed the test or not	Test results	Improved QoS or not
Eureka/service/registration and discovery	Service successfully registered	Passed	Successfully/registered to Eureka	Yes
Eureka service check	Remove fault service	Passed	successfully removed	Yes
Eureka Cluster	High availability	Passed	Eureka 7003 achieved	Yes
Zuul Gateway	Work normally	Passed	Gateway available	Yes
Feign	load balancing	Passed	DB sources different	Yes
Hystrix	Isolation	Passed	Cascading blocking	Yes
High availability	Achieved	Passed	Achieved	Yes
Docker container	Successful	Passed	successful	N/A

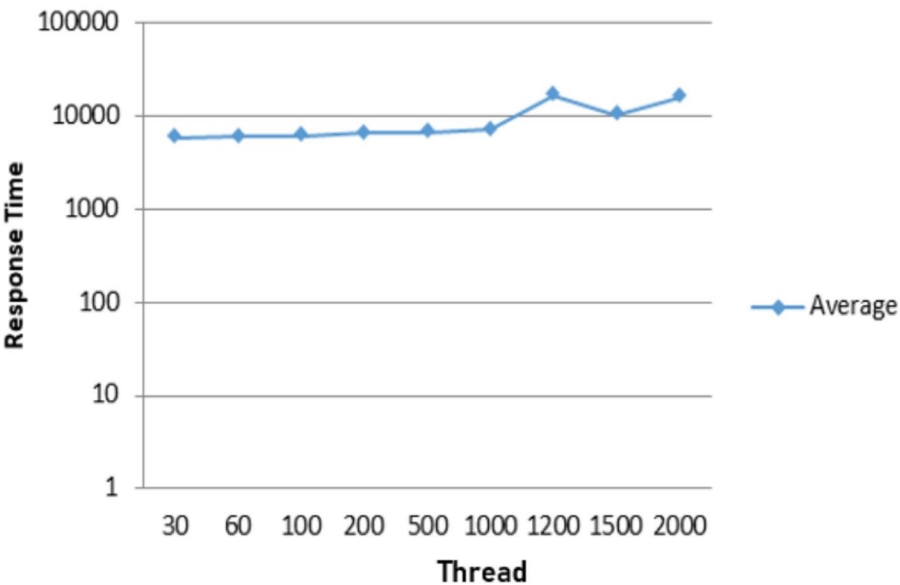


Fig. 12 Response timeline chart

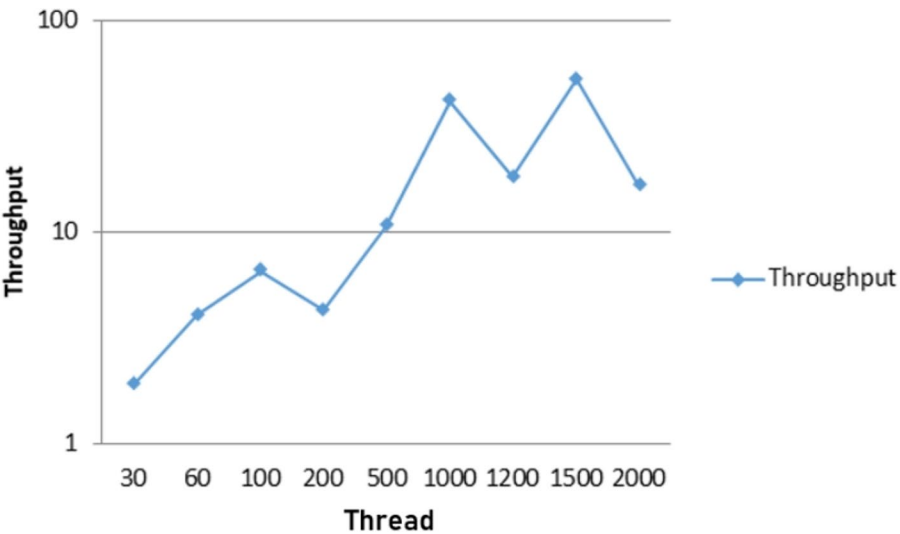


Fig. 13 Throughput chart

as the number of requests increased. The system demonstrated consistent low-latency performance, which is vital for real-time web applications.

(ii) **Throughput.**

Throughput means the number of requests completed per second by default, and sometimes it is also regarded as TPS. It can be easily seen from Fig. 13 that as the number of threads increases, the throughput increases accordingly. The architecture handled up to 1000 concurrent threads without significant degradation in performance, which showcases its scalability.

(iii) **99% Line.**

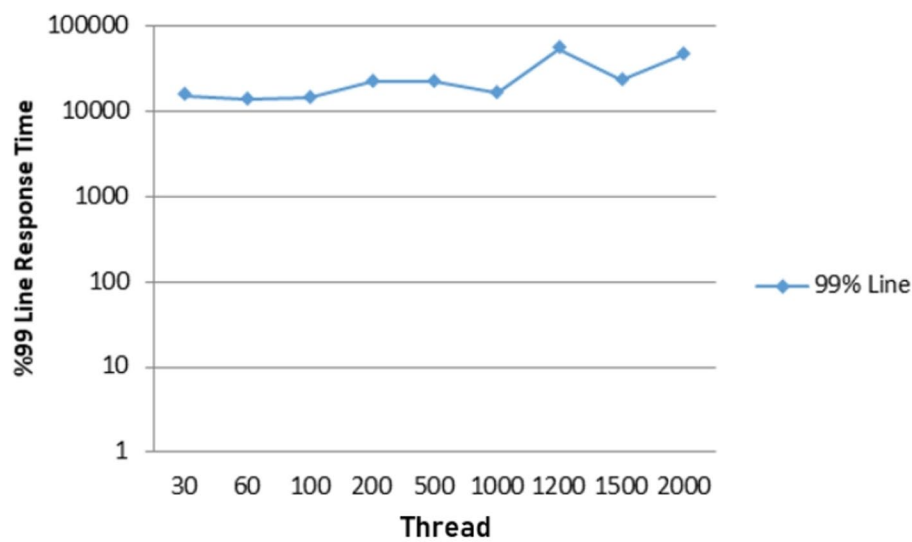


Fig. 14. 99% Line response timeline chart

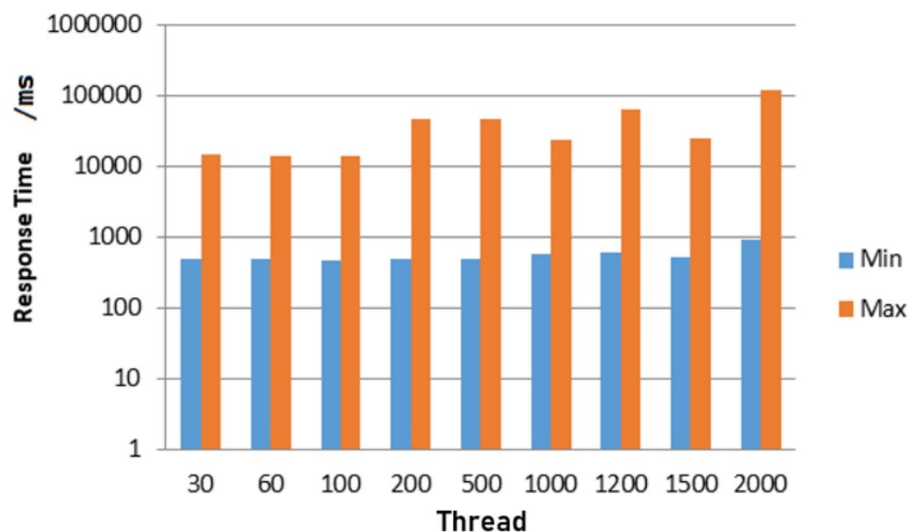


Fig. 15 Min and Max Response Timeline chart

99% Line is an indicator in the JMeter aggregation report. It refers to the response time of 99% of the requesting users. It is an important indicator of performance testing. Figure 14. shows that the response line is very flat; the test is as expected.

(iv) **Min and Max Response Time.**

The maximum and minimum response time is also essential indicator in the JMeter aggregation report. The maximum and minimum response time can demonstrate the ability of service response speed. It can be seen from Fig. 15 that the minimum response time is stable below 1000 ms, while the maximum response time reaches a maximum of 100,000 ms within the test range.

(v) **Received and Sent KB/sec.**

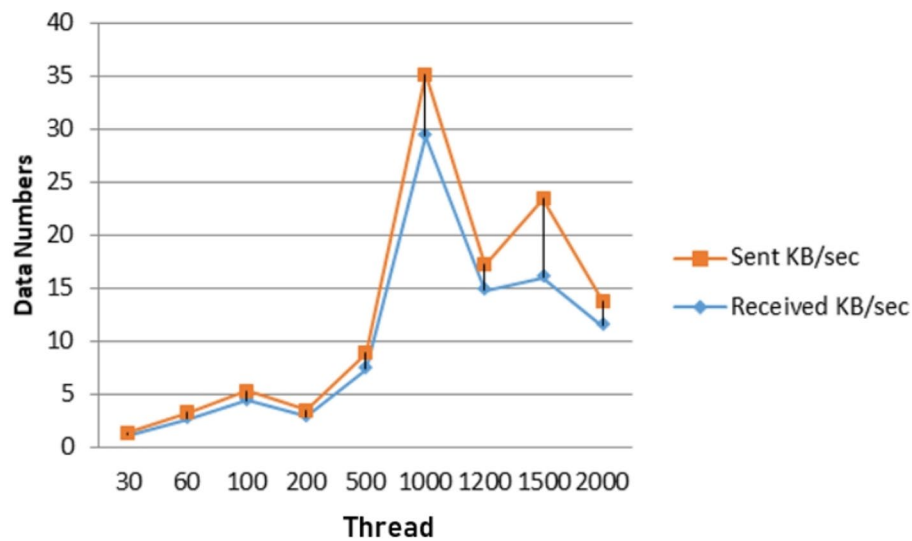


Fig. 16 Received and Sent KB/sec line chart

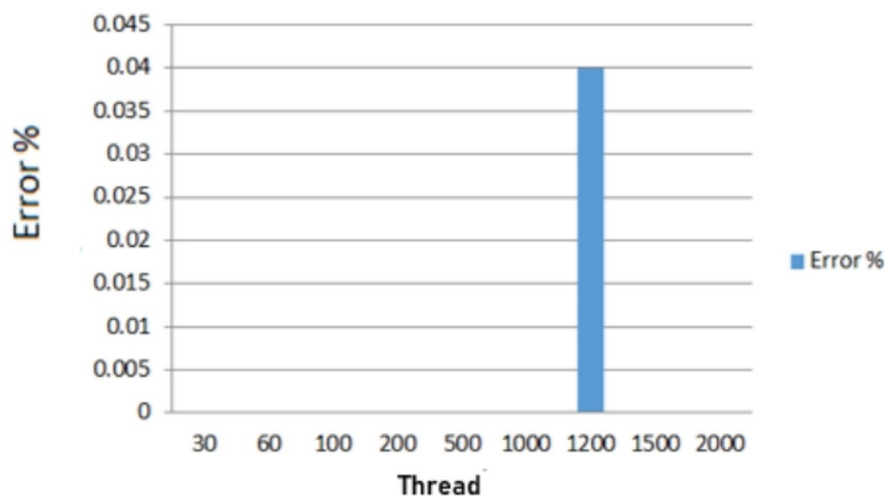


Fig. 17 Error % line chart

Received KB/sec represents the amount of data collected from the server per second and sent KB/sec is the number of requests sent from the client per second. As can be seen from the Fig. 16, it peaks at around 1000 threads, then it still drops and stays below 25. Due to the Alibaba server system environment, the data throughput may peak, so a small peak was caused at 1000 threads.

(vi) **Error %.**

Error% indicates the service's error request rate, which is used to measure the stability of the test service. As can be seen from the table, in the total number of all test threads, the error occurs only when the number of threads is 1200, and the error rate is 0.04. This error may be caused by an error in the HTTP request at a particular stage. Overall, as shown in Fig. 17, the system performance is relatively stable, and the error rate is controlled.

(vii) **JMeter Test Graphical Results.**

Figure 18 shows the results of the JMeter test result graph. It can be seen that the number of threads is 2000, the number of loops is 1, and the concurrency of threads is within a limited time. The JMeter configuration set as workload around 500–2000 threads, ramp-up time set as 60 s. Metrics: CPU/memory usage captured via Prometheus. The green curve in the figure shows that the throughput is stable, and the throughput is 2942.691/min. This test demonstrates the system's ability to manage high levels of traffic while maintaining consistent performance.

4.1.3 The analytic hierarchy process

The Analytic Hierarchy Process (AHP) was employed to evaluate Quality of Service (QoS) by weighting critical performance metrics. The criteria and weights were derived through pairwise comparisons by domain experts, ensuring alignment with industry standards for high-availability systems. If $CR < 0.$, the judgments are considered consistent. In our case, as we discussed in Sect. 3.6 all CR values are < 0.1 , indicating valid weight assignments.

4.2 Comparative study

This section has compared three different parts, the performance of microservices with and without the use of Spring Cloud components, the comparison of microservices with ZK and LAMP, and the comprehensive performance comparison of Microservices and Monolith.

4.2.1 Microservices and spring cloud components

The use of Spring Cloud's relevant components makes the system more robust and improves system performance. This is illustrated in Table 3.

4.2.2 Microservices, LAMP, and ZK comparison

This section compares the microservice architecture with the ZK architecture and the LAMP architecture. In comparison, the microservice architecture has improved the QoS of the system to a certain extent. Compared with monolithic architectures such as LAMP and ZK, microservices have excellent performance in many aspects. The comparison will be discussed in detail below:

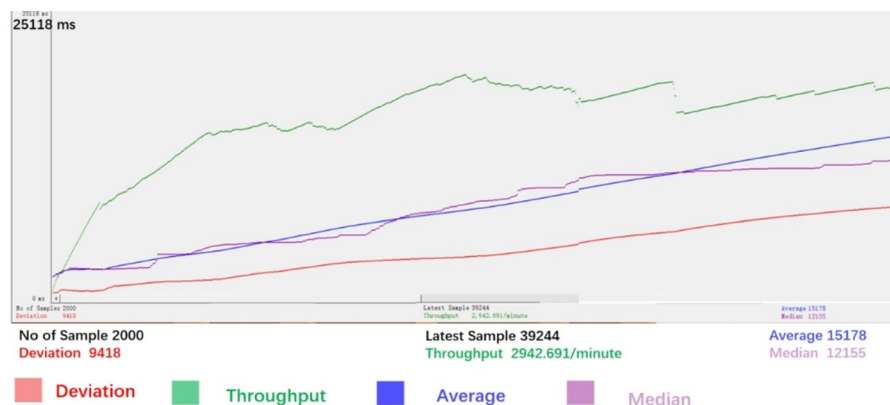


Fig. 18 JMeter concurrent test graphics results

Table 3 Microservices and spring cloud components

Criterion	Weight	Consistency ratio (CR)	Rationale
Response Time	0.35	0.08	Directly impacts user experience; prioritized for latency-sensitive systems
Throughput	0.30	0.06	Measures system capacity under load; critical for scalability
Error Rate	0.20	0.07	Reflects system stability; lower errors correlate with higher reliability
Resource Utilization	0.15	0.05	Ensures efficient use of CPU/memory; avoids over-provisioning costs

Table 4 Microservices and spring cloud components

Microservices components	QoS score	Robust (Yes/No)	Others
Spring Boot Only	0	No	Bad performance
Eureka	4	Yes	Good performance
Feign/Ribbon	3	Yes	Good performance
Cluster Function	3	Yes	Good performance

Table 5 Comparison between microservices and ZK

Parameters	ZK	QoS (score)	microservices	QoS (score)
Development difficulty	Medium	2	High	3
Stability	Low	1	Very high	4
Portability	Medium	2	High	3
Deployment dependencies	Very High	4	High	3
Service communication cost	Very High	4	High	3
Independent deployment	Low	1	Very high	4

(i) Comparison of Microservices and ZK.

Calculate the QoS score using the AHP-based QoS calculation method, and compare the characteristics of microservices and ZK architecture. Table 4 shows the comparison between microservices and ZK. Calculate the final ZK framework and microservices based on the data, ZK scores 0.58, and microservices is 0.83.

(ii) Comparison of Microservices and LAMP.

Similarly, a comparison with the LAMP architecture also showed that microservices offer significant QoS improvements. As shown in Table 5, the microservices architecture outperforms LAMP in terms of scalability, isolation, and independent deployment, with an overall QoS score of 0.85 compared to LAMP's 0.6.

4.2.3 A comprehensive comparison of monolithic and microservices

A broader comparison between monolithic and microservices architectures was conducted to evaluate key differences in performance and scalability. In Table 6, according to the characteristics of microservices and monomers in development and the differences in their respective functions, the differences and connections between microservices and monomers can be more intuitively displayed (Table 7).

From the comparison, it can be found that microservices are weak and single in terms of resource cost. However, in terms of independent deployment, scalability, and other characteristics, it can better help developers achieve business development. Overall, microservices have more good performance than monolithic architecture in many ways. The results of this evaluation confirm that the proposed microservices architecture

Table 6 Comparison of LAMP and microservices

Parameters	LAMP	QoS (Score)	Microservices	QoS (score)
Open-source	Very high	4	Very high	4
Multiple components	High	3	High	3
Rapid development	Medium	2	High	3
High performance	Medium	2	High	3
Cross-platform	Very High	4	Very high	4
Program readability	Medium	2	High	3
Independent deployment	Medium	2	Very high	4
Technology Stack	Low	1	High	3
Isolation	Medium	2	Very high	4
Expandability	Medium	2	High	3

Table 7 Comprehensive comparison of monolith and microservices

Functions	Monolithic	Microservices	Better/performance/choice
Difficulty of Splitting	Higher	Lower	Microservices
Scalability	Limited	Flexible	Microservices
Independent Deployment	Limited	Flexible	Microservices
Remote Call	Limited	Flexible	Microservices
O&M Complexity	Higher	Lower	Monolith
Reusability	Lower	Higher	Microservices
Resource Cost	Lower	Higher	Monolith
Communication Efficiency	Lower	Higher	Microservices
Technology Stack	Lower	Higher	Microservices
Tool Dependency	Lower	Higher	Monolith
Developer Requirements	Lower	Higher	Monolith
Service Governance	Limited	Flexible	Microservices

significantly improves QoS in web systems. The system demonstrated superior performance in terms of throughput, response time, and fault tolerance compared to traditional monolithic architectures. The high availability strategies implemented, including Eureka clusters and Docker containers, further enhanced system reliability and deployment efficiency.

5 Conclusion

Microservices architecture is a fundamental concept that has emerged in recent years. On the one hand, monolithic architecture gradually fails to keep pace with the development of web systems. On the other hand, with the continuous development of microservice architecture and container technology, the system architecture of many SMEs will move towards the path of microservices. Therefore, microservices will become more and more popular in the future. However, monolithic architecture is facing a series of problems, such as low development efficiency, and poor system isolation. To solve the issues of the monolithic architecture and improve the performance of the system. This paper also has some limitations like single-server testing, in the future work includes multi-node experiments, also for industry relevance, need aligns with DevOps practices for CI/CD and cloud-native deployments. This paper proposes a robust microservices architecture and successfully improves the QoS of the web system through a series of service management components and high availability strategies. The research topic is about the improvement of the QoS of web systems by the microservices architecture and building a specific microservices project, obtaining the system performance data

through experiments, and successfully proving the feasibility and reliability of the architecture. The specific work of this paper is summarized as follows:

1. The evolution of the system architecture is discussed, and the microservices, Spring Boot, and Spring Cloud that have appeared in recent years are introduced in detail. Through research and comparison, Spring Cloud was selected to build a microservices system, and Eureka, Feign, Hystrix, and Zuul were chosen as the main services management components.
2. To study the configuration and operation principles of the microservices framework Spring Boot and apply them to the implementation of this microservices project. Researched the main components of the Spring Cloud framework and built Eureka as service discovery and registration, Feign as a load balancer, Hystrix as a service fuse, and Zuul as a routing gateway.
3. To simulate the application of microservices more comprehensively, the system is divided into four Spring Cloud component services and three application services in the project. The four Spring Cloud component services implement service governance. The three application services, userService, userOrderService and productService, can be independently deployed, independently maintained, and individually expanded, and services can communicate with each other through a unified protocol.
4. To improve the QoS of the system, this paper designs and implements three high availability strategies: Eureka Health Check, Eureka Cluster, and Application Service Cluster. Through the high availability strategy, high availability and high reliability of microservices are achieved, and finally, the QoS of the system is successfully improved.
5. To deploy the MySQL database on the purchased Alibaba Cloud server, the Dockerfile of each service was written. The microservices project was deployed into the Docker container, which realized the rapid deployment of microservices. After deployment, the microservices project was comprehensively tested through functional and performance tests, and a large amount of performance test data was obtained.

According to the test results, the microservices architecture system has a QoS Score of more than 20% higher than the ZK and LAMP architecture system, which means the robust microservices architecture can improve the QoS of the web system. Although the microservices project designed in this paper has passed the final system test, the project can only meet simple business requirements and performance. In actual large-scale microservices projects, to achieve a more robust architecture, more factors need to be considered. Therefore, the next step may be to consider deploying an automated application deployment system instead of manual deployment. The microservices project in this paper does not carry out large-scale node deployment. Future work could explore automated deployment systems for large-scale microservices projects, enhancing the architecture's resilience and efficiency. Additionally, large-scale node deployments and more advanced load-balancing strategies could be examined to further optimize the system's performance in complex, real-world environments.

Author contributions

Pan Zhang, Yuhong Yang, Zhuo Song, and Lixia Xiang contributed to the conceptualization and design of the study. Pan Zhang was responsible for methodology development, software implementation, and experimental execution. Yuhong Yang conducted data collection, experimental validation, and prepared the original manuscript draft. Zhuo Song performed data analysis, visualization, and contributed to manuscript review and editing. Lixia Xiang supervised the project and managed the typesetting of the manuscript. Pan Zhang and Lixia Xiang acquired funding for the study. All authors have reviewed and approved the final version of the manuscript.

Funding

This work was supported by the Guangxi Higher Education Young and Middle-aged Teachers' Research Basic Ability Enhancement Project (Research and Design of a Vocational Education Discipline Ability Assessment System Based on Intelligent Evaluation, 2024KY2019), and the Dayao Mountain Unity Covenant Special Project (Innovative Research on Ethnic Education Driven by Digital Technology under the "Dayao Mountain Unity Covenant", GXKS2023QN067).

Availability of data and materials

The data supporting the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Clinical trial number

Not applicable.

Received: 24 February 2025 / Accepted: 12 June 2025

Published online: 01 July 2025

References

1. Richardson C (2017) Pattern: monolithic architecture. Available: <http://microservices.io/patterns/monolithic.html>
2. Dragoni N, Giallorenzo S, Lafuente AL, Mazzara M, Montesi F, Mustafin R, Safina L (2017) Microservices: yesterday, today, and tomorrow. In: *resnet and ulterior software engineering*, pp 195–216. Springer
3. Peng H (2011) Design and implementation of data nodes for mass storage systems. Shanghai Jiaotong University
4. Mazlami G (2017) Algorithmic extraction of microservices from monolithic code bases. Master Paper, Software Evolution and Architecture Lab, Department of Informatics, University of Zurich
5. Fowler SJ (2016) *Production-ready microservices: building standardized systems across an engineering organization*. O'Reilly UK Ltd
6. Mazlami G, Cito J, Leitner P (2017) Extraction of microservices from monolithic software architectures. In: *IEEE 24th international conference on web services (ICWS)*, pp 524–531
7. Erl T (2005) *Service-oriented architecture: concepts, technology, and design*. Indiana: Prentice Hall PTR
8. Endrei M, Ang J, Arsanjani A, Chua S, Comte P, Kroghdahl P, Luo M, Newling T (2004) *Patterns: service-oriented architecture and web services*. In: IBM corporation, international technical support organization
9. Claus L, Pooyan J (2016) Microservices: a systematic mapping study. In: *Proceedings of the 6th international conference on cloud computing and services science*, pp 137–146
10. Newman S (2015) *Building microservices*. In: Loukides M, MacDonald B (eds), 1st edn. O'Reilly Media, Inc.
11. Beyer B, Jones C, Petroff J, Murphy NR (2016) *Site reliability engineering: how google runs production systems*, 1st edn. O'Reilly
12. Kai X. Design and implementation of training business system based on spring cloud. Beijing: Beijing University of Posts and Telecommunications; 2019.
13. Gamma E, Helm R, Johnson R, Vlissides J. *Design patterns: elements of reusable object-oriented software*. Boston: Addison Wesley; 1994.
14. Reddy KSP (2017) *Spring boot autoconfiguration. Beginning Spring Boot 2*
15. Carnell J. *Spring Microservices in action*. Suffolk County. New York: Manning Publications; 2017.
16. Dong N. Research on web system performance tests. Guangzhou: Jinan University; 2008.
17. Buceamaneatonis R. How to design a web survey using spring boot with MYSQL: a Romanian network case study. *Soc Sci Electron Publ*. 2017;17(2):63.
18. Gammelgaard CH (2016) *Microservices in .NET Core*. Shelter Island, NY: Manning
19. Liu X. The Design and Implementation of the Web development framework of the software system based on the MVC model. Shandong: Shandong University; 2018.
20. Lin RX. Research and development of an attendance management system based on the C/S and B/S mixed model. *Neijiang Sci Technol*. 2007;28(7):135–135.
21. Wang CG, Lu JG. WEB development model and its application based on SPRING and VELOCITY. *Microcomput Inf*. 2006;22(30):230–2.
22. Sang YM. Architecture of high performance web server based on LAMP. *Intell Comput Appl*. 2011;6(6):44–6.
23. Wang LL. Research and optimization of high-performance web application system architecture. Beijing: Beijing University of Posts and Telecommunications; 2016.
24. Sinz C, K hosravizadeh A, Wolfgang K, et al (2003) Verifying CIM models of Apache web-server configuration. In: *Quality software, international conference on IEEE computer society*, pp 290–290
25. Liu SY, Li Q, Li B. Research on container isolation based on Docker technology. *Software*. 2015;4:110–3.
26. Tokmak AV, Akbulut A, Catal C. Boosting service visibility in microservices. *Clust Comput*. 2024;2024:1234567.
27. Madnani N, Cahill A, Blanchard D, Andreyev S, Napolitano D, Gyawali B, Heilman M, Lee CM, Leong CW, Mulholland M, Riordan B. A robust microservice architecture for scaling automated scoring applications. *ETS Res Rep Ser*. 2018;2018:1–8.

28. Bhamare D, Samaka M, Erbad A, Jain R, Gupta L. Exploring microservices for enhancing internet QoS. *Trans Emerg Tel Tech*. 2018;29: e3445.
29. Al-Masri E (2018). Enhancing the microservices architecture for the Internet of Things. In: 2018 IEEE international conference on big data (Big Data), Seattle, WA, USA, pp 5119–5125
30. Bushong V, Abdelfattah AS, Maruf AA, Das D, et al. On microservice analysis and architecture evolution: a systematic mapping study. *Appl Sci*. 2021;11(17):7856.
31. Gao KL. Analysis and research on the related technology of web performance. *Modern Electron Technol*. 2003;11(11):46–58.
32. Söylemez M, Tekinerdogan B, Kolukisa TA. Challenges and solution directions of microservice architectures: a systematic literature review. *Appl Sci*. 2022;12(11):5507.
33. Samodha P, Vassilis K, Rajkumar B. QoS-aware placement of microservices-based IoT applications in Fog computing environments. *Futur Gener Comput Syst*. 2022;131:121–36.
34. Hossen MR, Islam MA, Ahmed K (2022) Practical efficient microservice autoscaling with QoS assurance. In: Proceedings of the 31st international symposium on high-performance parallel and distributed computing (HPDC '22). Association for Computing Machinery, New York, pp 240–252
35. Tokmak AV, Akbulut A, Catal C. Boosting the visibility of services in microservice architecture. *Cluster Comput*. 2024;27:3099–111.
36. Hassaan S, Ferhat K, Maria T. Microservices based architectures for IoT systems—state-of-the-art review. *Internet of Things*. 2023;23: 100854.
37. Alelyani A, Datta A, Hassan GM. Optimizing cloud performance: a microservice scheduling strategy for enhanced fault-tolerance, reduced network traffic, and lower latency. *IEEE Access*. 2024;12:35135–53.
38. Singh N, Hamid Y, Juneja S, et al. Load balancing and service discovery using Docker Swarm for microservice based big data applications. *J Cloud Comp*. 2023;12(4):1. <https://doi.org/10.1186/s13677-022-00358-7>.
39. Hossain MD, Sultana T, Akhter S, et al. The role of microservice approach in edge computing: opportunities, challenges, and research directions. *ICT Express*. 2023;9(6):1162–82.
40. Dashtbani M, Tahvildari L (2025). STaleX: a spatiotemporal-aware adaptive auto-scaling framework for microservices. <http://arxiv.org/abs/2501.18734>. <https://doi.org/10.48550/arXiv.2501.18734>
41. Barua B, Kaiser MS (2024) Enhancing resilience and scalability in travel booking systems: a microservices approach to fault tolerance, load balancing, and service discovery. <http://arxiv.org/abs/2410.19701>. <https://doi.org/10.48550/arXiv.2410.19701>
42. Bonr J (2016) Reactive microservices architecture. O'Reilly Media, Inc. Available: <http://proquest.tech.safaribooksonline.de/9781491975664>
43. Brewer E. Pushing the CAP: strategies for consistency and availability. *Computer*. 2002;45(2):23–9.
44. Lan XH, Xiong JJ, Deng G. Application programming based on MYSQL. *Comput Eng Des*. 2004;25(3):442–3.
45. Taibi D, Lenarduzzi V, Pahl C, et al (2017). Microservices in agile software development: a workshop-based study into issues, advantages, and disadvantages. In: Xp2017 scientific workshops. ACM, vol 23
46. Yang XR, Zhu GJ, Yang H, et al. Comprehensive evaluation of the warehouse distribution system of Shanghai Mingdao Trading Company. *Sci Technol Inf*. 2019;18:118–9.
47. Rathore M, Suman U. (2013). Evaluating QoS parameters for ranking web service. In: Advance computing conference (IACC), 2013 IEEE 3rd international IEEE, pp 1437–1442.
48. Kayastha RR, Baria J. A survey on web service selection and ranking approach. *Int J Comput Appl*. 2015;117(16):5–8.
49. Xiong K (2019) Design and implementation of training business system based on spring cloud. Beijing University of Posts and Telecommunications, Beijing
50. Balalaie A, Heydarnoori A, Jamshidi P (2015) Microservices migration patterns. Tech.Rep.TR-SUTCE-ASE-2015-01, Automated Software Engineering Group.Tehran.Iran: Sharif University of Technology
51. Leprevost FD, Gruning BA, Alvers AS. BioContainers: an open-source and community-driven framework for software standardization. *Bioinformatics*. 2017;33(16):2580–2.
52. Merkel D. Docker: lightweight Linux containers for consistent development and deployment. *Linux J*. 2014;239:2.
53. Su CJ, He P. Research on the application of PHP in web database. *Comput Eng*. 2000;09(09):184–5.
54. Wolff E (2015) Microservices: Grundlagen flexible Softwarearchitekturen, 1st ed. dpunkt. Verlag GmbH
55. Zeljko J, Jagodic D, Vujicic D, et al (2017) Java spring boot rest WEB service integration with java artificial intelligence Weka framework. In: UNITECH 2017, International Scientific Conference, Gabrovo
56. Zhang L. The college morning exercises attendance monitoring system based on ZK framework. Tianjin: Tianjin University; 2017.
57. Zheng MZ, Zhang JQ. Discussion on the evolution of micro-service-based system architecture. *Software*. 2017;38(12):165–9.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.